32. Design Xbox Live Leaderboard

Key Terms:

Term	Full Name	What it does
ZADD	Redis Sorted Set Add	Adds member with score to sorted set
ZRANK	Redis Sorted Set Rank	Gets rank of member (0-indexed)
ZREVRANGE	Redis Reverse Range	Gets members by rank, highest first
Sorted Set	Redis data structure	Collection sorted by score, O(log N) operations
TTL	Time To Live	Auto-expiration for time-based leaderboards
Anti-cheat	Score validation	ML/rules to detect invalid scores

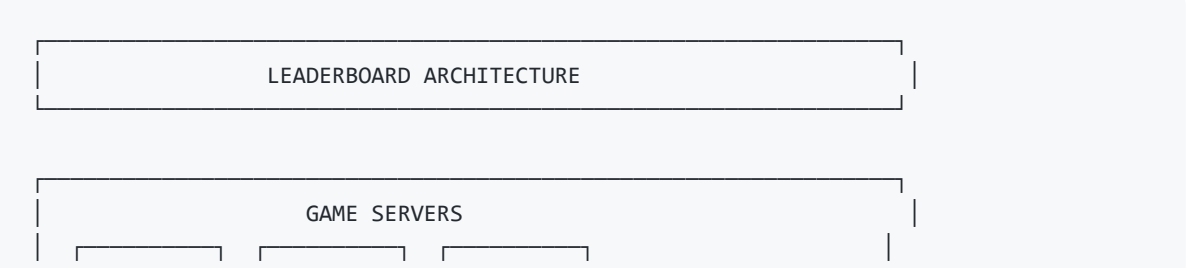
Problem Statement

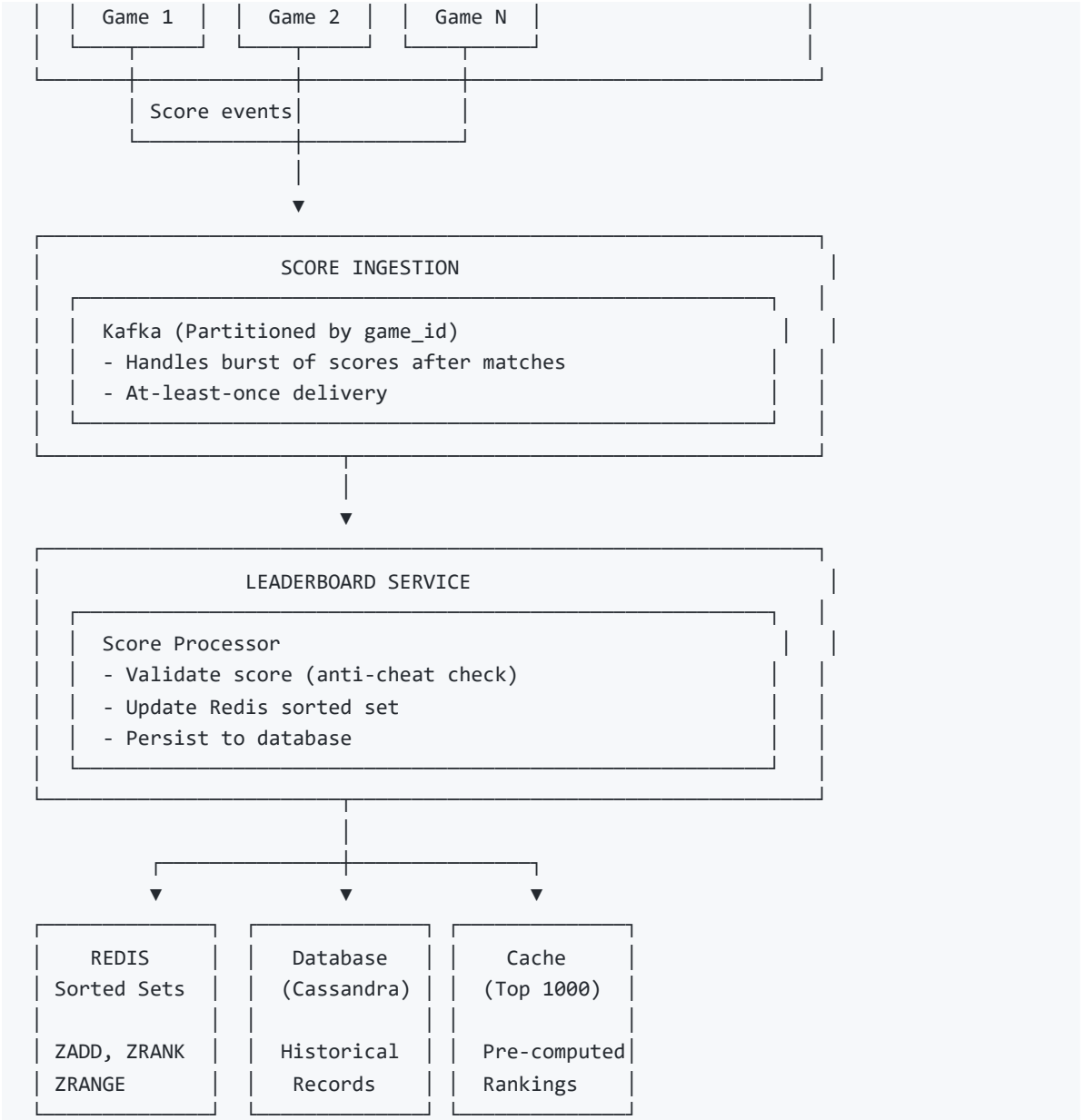
Design a global leaderboard system for Xbox Live that ranks millions of players in real-time across multiple games and categories.

Clarifying Questions to Ask

1. **Scale**: How many players? Games? (100M players, 1000 games?)
2. **Update frequency**: Real-time or periodic updates?
3. **Leaderboard types**: Global, friends, regional, time-based?
4. **Query patterns**: Top N, rank of specific player, nearby ranks?
5. **Score updates**: Frequency per player? (every match?)
6. **Historical**: Keep historical leaderboards? (weekly, monthly?)
7. **Cheating**: How to handle cheating/invalid scores?

High-Level Design





API Design

Leaderboard APIs

Method	Endpoint	Description
`GET`	`/api/v1/leaderboards/{boardId}/top?limit=100`	Get top players
`GET`	`/api/v1/leaderboards/{boardId}/around/{userId}?range=10`	Get players around a user
`GET`	`/api/v1/leaderboards/{boardId}/rank/{userId}`	Get user's rank and score

`GET`	`/api/v1/leaderboards/{boardId}/friends/{userId}`	Get friends leaderboard

Score Submission APIs

Method	Endpoint	Description
`POST`	`/api/v1/scores`	Submit a score
`GET`	`/api/v1/scores/{userId}/history?game={id}`	Get user's score history

Submit Score Request

```
POST /api/v1/scores
{
  "user_id": "player_123",
  "leaderboard_id": "halo_weekly",
  "score": 15000,
  "metadata": {
    "game_id": "match_456",
    "level": 42,
    "timestamp": 1672531200
  }
}

Response: 200 OK
{
  "new_rank": 42,
  "previous_rank": 55,
  "percentile": 98.5
}
```

Leaderboard Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/leaderboards`	Create a leaderboard
`POST`	`/api/v1/leaderboards/{boardId}/reset`	Reset leaderboard (season end)

Key Components

Component	Purpose	Technology
Score Ingestion	Buffer score updates	Kafka

Leaderboard Service	Process and rank	Custom service
Real-time Rankings	Fast rank queries	Redis Sorted Sets
Persistence	Historical data	Cassandra
Cache	Top N pre-computed	Redis/CDN
Anti-cheat	Validate scores	ML anomaly detection

Redis Sorted Set Operations

```
REDIS COMMANDS FOR LEADERBOARD:
┌─────────────────────────────────────────────────────────────────┐
│  # Update player score                                          │
│  ZADD leaderboard:halo:weekly 15000 player_123                  │
│                                                                 │
│  # Get player's rank (0-indexed, so add 1)                      │
│  ZREVRANK leaderboard:halo:weekly player_123                    │
│  → 42 (meaning rank #43)                                        │
│                                                                 │
│  # Get top 10 players with scores                               │
│  ZREVRANGE leaderboard:halo:weekly 0 9 WITHSCORES               │
│  → [player_1, 50000, player_2, 48000, ...]                      │
│                                                                 │
│  # Get players around a specific rank (rank 40-50)              │
│  ZREVRANGE leaderboard:halo:weekly 39 49 WITHSCORES             │
│                                                                 │
│  # Get player's score                                           │
│  ZSCORE leaderboard:halo:weekly player_123                      │
│  → 15000                                                        │
│                                                                 │
│  # Count total players                                          │
│  ZCARD leaderboard:halo:weekly                                  │
│  → 5000000                                                      │
└─────────────────────────────────────────────────────────────────┘

Time Complexity:
  ZADD:     O(log N)
  ZRANK:    O(log N)
  ZREVRANGE: O(log N + M) where M is result size
```

Leaderboard Types

```
1. GLOBAL LEADERBOARD (All players)
   Key: leaderboard:{game}:global

2. FRIENDS LEADERBOARD
   - Fetch friend list
```

```
   - ZSCORE for each friend
   - Sort client-side (small list)

3. TIME-BASED LEADERBOARDS
   Keys: leaderboard:{game}:daily:{date}
         leaderboard:{game}:weekly:{week}
         leaderboard:{game}:monthly:{month}

   Reset: Scheduled job creates new key, archives old

4. REGIONAL LEADERBOARDS
   Keys: leaderboard:{game}:{region}
   Sharding by region reduces single key size
```

Scalability Considerations

- **Sharding**: Separate Redis instances per game or region
- **Pre-computation**: Cache top 1000 with short TTL
- **Batch updates**: Aggregate scores before writing
- **Archival**: Move old leaderboards to cold storage
- **Eventually consistent**: Rank updates can be slightly delayed

Trade-offs

Decision	Option A	Option B	Recommendation
Storage	SQL	Redis Sorted Set	**Redis** - O(log N) rank
Update	Real-time	Batched	**Batched** - handle spikes
Friends LB	Pre-compute	On-demand	**On-demand** - friend lists change
Historical	Keep all	Sample	**Archive + sample**

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Sorted sets for O(log N) rank operations	Tech Guide
Kafka	Score ingestion and event streaming	Tech Guide
Cassandra	Historical leaderboard persistence	Tech Guide

How to Explain It Well

> **Opening**: "Leaderboards need fast rank lookups for millions of players. I'll use Redis Sorted Sets for O(log N) operations."

> **Key talking points**:
>
> - "Redis ZREVRANK gives player's rank in O(log N)"
> - "Shard by game/region to distribute load"
> - "Time-based leaderboards: new key per period, archive old"
> - "Friends leaderboard: fetch scores on-demand, sort client-side"
> - "Batch score updates during high traffic (end of match)"
